

CMU-CS 462: Software Measurement and Analysis

Spring 2025-2026

Nguyen Duc Man
E: mannd@duytan.edu.vn
T: 0904235945

1

Outline of The Course

Part 1: Measurement theory

Part 2: Software product, process and project measurements

Part 3: Measurement management



2

2

Overview of Software Metrics

Nguyen Duc Man
E: mannd@duytan.edu.vn
T: 0904235945

3

Contents

1. What is measurement?
2. What are software metrics?
3. Scope of software engineering metrics:
a chronological review.



10

4

Measurement: Definition /1

- **Measurement** is the process by which **numbers** or **symbols** are assigned to **attributes** of **entities** (**objects**) in the real world in such a way as to ascribe them according to defined rules.

Object

Đo lường là quá trình mà các số hoặc ký hiệu được gán cho các thuộc tính của các thực thể (đối tượng) trong thế giới thực theo cách để mô tả chúng theo các quy tắc xác định.

5

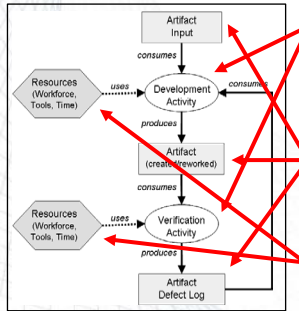
Measurement: Definition /2

- **Metrics** are standards (i.e., commonly accepted scales) that define measurable attributes of entities, their units and their scopes.
- **Measure** is a relation between an attribute and a measurement scale.

6

Entity

- An entity in software measurement can be any of the following:



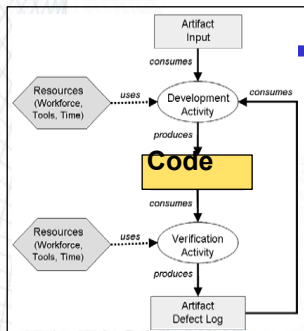
7

Attribute

- An **attribute** is a feature or property of an entity
 - e.g., blood pressure of a person, cost of a journey, duration of the software specification process

- There are two general types of attributes:

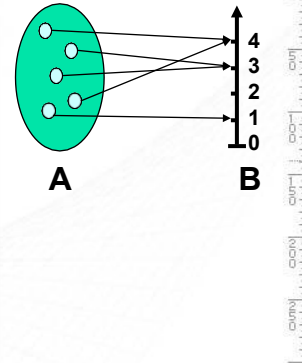
- **Internal attributes** can be measured only based on the entity itself,
 - e.g., entity: code, internal attribute: size, modularity, coupling
- **External attributes** can be measured only with respect to how the entity relates to its environment
 - e.g., entity: code, external attribute: reliability, maintainability



8

Measurement Example

Entity	Attribute
Requirements	Size, Reuse, Redundancy
Specification	Size, Reuse, Redundancy
Design	Size, Reuse, Modularity, Cohesion, Coupling
Code	Size, Reuse, Modularity, Cohesion, Coupling, Complexity
Test Cases	Size, Coverage



9

Measurement: Types

- Measurements are needed as:
 - **Descriptors** of entities already in existence.
 - **Prescriptors** (standards, norms, failure intensity objectives, benchmarks) which entities of certain class or category should satisfy.
 - **Predictors** to estimate properties of entities yet to be designed or implemented.

10

10

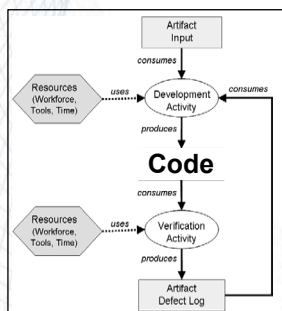
Measurement : How to

- In order to make entities measurable:
 - What **entities (objects)** should be selected?
 - What **attributes** should be selected?
 - What **values** should be assigned to the attributes?
 - What shall be the **rules (relationships)** ascribed to the attributes and their entities?
- **Note:** assigned values and/or ascribed rules can be either **quantitative** or **qualitative**.

11

11

Example 2: Code



- **Entity: Code**
- **Attribute: Size**
- **Possible measures:**
 - NCSLOC (Not Commented Source Lines of Code)
 - #Statements
 - #Modules
 - #Procedures
 - #Classes
 - #Methods
 - ...



12

12

Example 3: Availability

- **Entity:** Availability
- **Attributes:** system uptime, system downtime
- **Values:** time in seconds
- **Relations:**

$$\text{Availability} = \text{uptime} / (\text{uptime} + \text{downtime})$$

22

13

Software Metrics Challenges

- Measuring physical entities:

<u>entity</u>	<u>attribute</u>	<u>unit</u>	<u>value</u>
Human	Height	cm	178

- Measuring non-physical entities:

<u>entity</u>	<u>attribute</u>	<u>unit</u>	<u>value</u>
Human	IQ	IQ index	89

- SE metrics are mostly non-physical
 - Reliability, maturity, portability, flexibility, maintainability, etc. and relations are unknown

14

14

Misleading Metrics!

- Fact (1): Knowledge is power
- Fact (2): Time is money
- Relation (rule): $\text{power} = \text{work} / \text{time}$

Substituting “power” and “time”

- Knowledge = work / money
- As knowledge approaches zero money approaches infinity regardless of the amount of work done!
- Conclusion:
 - The less you know, the more you make!



What went wrong here?

- Counter intuitive
- Needs validation

15

15

What Is Software Measurement?

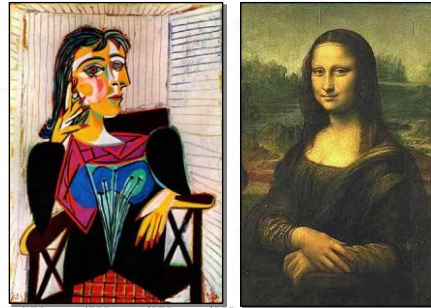
- Software metrics are measures that are used to quantify software, software development resources, and/or the software development process.
- This includes items which are directly measurable, such as *lines of code*, as well as items which are calculated from measurements, such as *software quality*.

16

16

Measurement in SE

- Measurement in SE is selecting, measuring and putting together many different attributes of the software, and adding our subjective interpretations in order to get a whole picture of the software.
- This is not a trivial task!
- 300+ metrics have been defined.



17

17

Measurement in SE /1

- Before a measurement project can be planned
 - Objectives and scope should be established
 - Alternative solutions should be considered
 - Technical and management constraints should be identified.
- This information is required to estimate costs, project tasks, and a project schedule.

18

18

Measurement in SE /2

- In order to manage software measurement project one must understand and plan:
 - The goal and scope of work
 - Risks
 - Resources required
 - Tasks to be accomplished
 - Milestones to be tracked
 - Total costs of the project
 - Schedule to be followed

19

19

Measurement in SE /3

- Software metrics help us understand the technical process that is used to develop a software product.
 - The process is measured to be improved.
 - The product is measured to increase its quality.

But ...

- Measuring software projects is controversial.
- It is not yet clear which are the appropriate metrics for a software project or whether people, processes, or products can be compared using metrics.

20

20

Scope of Software Metrics

- Cost and effort estimation
- Productivity measures and models
- Data collection
- Quality models and measures
- Reliability models
- Performance evaluation and models

**More about
scope in
the next
session**

21

21

Scope of Software Metrics

- Performance evaluation and models
- Structural and complexity metrics
- Capability maturity assessment
- Management by metrics
- Evaluation of methods and tools

**More about
scope in
the next
session**

22

22

Example 1

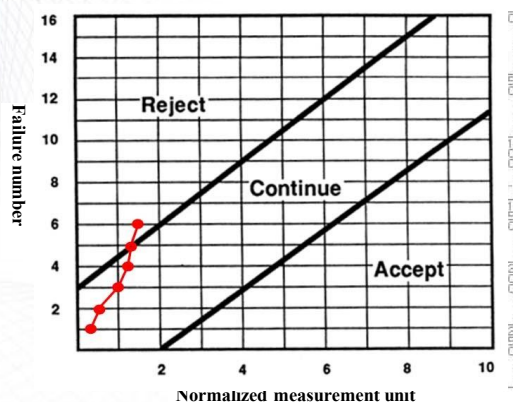
- We are going to buy a new colour laser printer for our department. We have borrowed the printer for the test.
- Maker's data shows that we need to change the toner every 10,000 pages. We would like to have only one failure during the period.
- During test period, we observe that failures occur at 4,000 pages, 6,000 pages, 10,000 pages, 11,000 pages, 12,000 pages and 15,000 pages of output.
- What can we conclude about this printer?

33

23

Example 1 (cont'd)

- Failure intensity objective:
 $\lambda_F = 1/10000$ pages
- Using reliability demonstration chart we can conclude that the printer must be rejected.



34

24

Example 2

- We are going to initiate a new game and video on-demand download service. The service is provided to the customers who own PCs and register with the service.
- The customers must use specialized software to download games or videos from the server. The failure intensity of the software is 1 failure per 100 CPU hr. On average, the specialized software system runs 20 CPU hr per week on each client machine and there are 800 customers to be serviced.
- We would like to provide the customers with an on-site repair service. Each serviceperson can make 4 service calls per day. Service personnel are available 5 working days per week.
- How many service personnel do we need to hire?

35

25

Example 2 (cont'd)

- How many service personnel do we need?
- Using the value for failure intensity, each system experiences 0.2 failure per 20 hours of operation or 0.2 failure per week, on average.
- The total failures for 800 customers is 160 per week or 32 per day.
- Each serviceperson can visit 4 sites per day, therefore, the number of required personnel is $32 / 4 = 8$.

36

26



Overview: Scope of Software Metrics

Process, Product and Resources

27



Scope of Software Metrics

- Cost and effort estimation
- Productivity measures and models
- Data collection
- Quality models and measures
- Reliability models
- Performance evaluation and models

28

39

Scope of Software Metrics

- Performance evaluation and models
- Structural and complexity metrics
- Capability maturity assessment
- Management by metrics
- Evaluation of methods and tools

39

29

Scope of Software Metrics/1

Cost and effort estimation

- Software cost estimation is the process of predicting the amount of effort required to build a software system.
- Estimates for project cost and time requirements are derived during the planning stage of a project.
- Models used to estimate cost can be categorized as either cost models (e.g., Constructive Cost Model COCOMO) or constraint models (e.g., SLIM).
- Experience is often the only guide used to derive these estimates, but it may be insufficient if the project breaks new ground.
- Many models are available as automated tools.

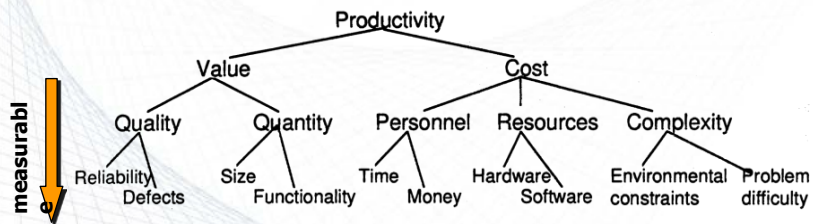
40

30

Scope of Software Metrics/2

Productivity models and measures

- **Definition:** The rate of output per unit of input.
 - Productivity = size / effort
 - Productivity = LOC / person-month
- Productivity model based on decomposition to measurable attributes:



41

31

Scope of Software Metrics/3

Data collection

- Very critical and very hard step.
 - What data should be collected?
 - How it should be collected?
 - Is collected data reproducible?
- **Example:** software failure data collection
 - 1) Time of failure
 - 2) Time interval between failures
 - 3) Cumulative failure up to a given time
 - 4) Failures experienced in a time interval

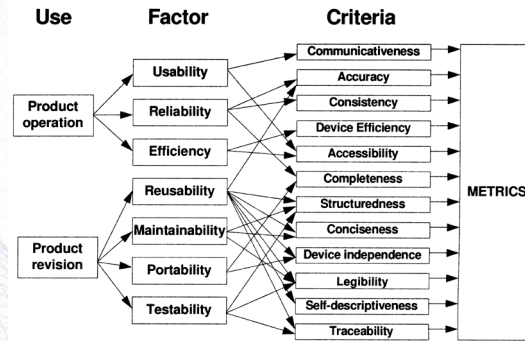
42

32

Scope of Software Metrics/4

Quality models and measures

- Software quality measurement (Rubey and Hartwick 1968~)
- McCall's quality factors (1977~), ISO 9126



43

33

Scope of Software Metrics/5

Reliability models

- Plot the change of failure intensity (λ) against time.
- Many models are proposed. The most famous ones are **basic exponential model** and **logarithmic Poisson model**.
- The basic exponential model assumes finite failures in infinite time; the logarithmic Poisson model assumes infinite failures.
- Automated tools such as CASRE are available.

44

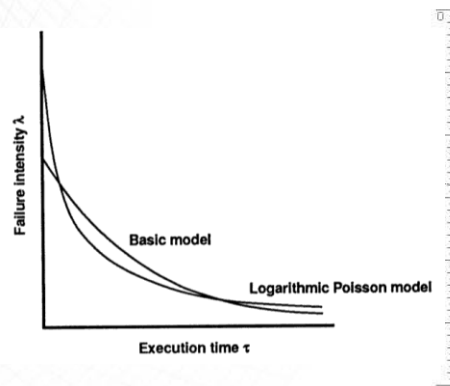
34

Reliability Models

- Failure intensity (λ) versus execution time (τ)

$$(B) \quad \lambda(\tau) = \lambda_0 e^{\left(-\frac{\lambda_0}{V_0}\right)\tau}$$

$$(P) \quad \lambda(\tau) = \frac{\lambda_0}{\lambda_0 \theta \tau + 1}$$



45

35

Scope of Software Metrics/6

Performance evaluation and models

- Using externally observable performance characteristics such as response time and completion rate
- Efficiency of algorithms

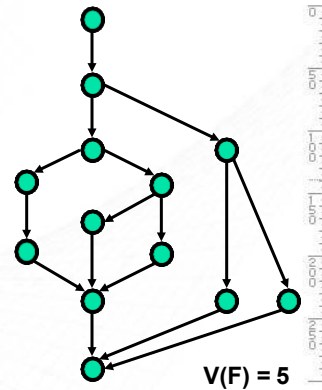
46

36

Scope of Software Metrics/7

Structural and complexity metrics

- Control-flow structure
- Data-flow structure
- Data structure
- Information flow attributes
- Complexity metrics
 - Cyclomatic complexity (McCabe 1989)
defining number of independent paths in execution of a program



37

Scope of Software Metrics/8

Management by metrics

- Metrics for project control
 - Metrics related to specification quality
 - Metrics for the design model
 - Metrics for documentation
 - Checking and testing metrics
 - Resource metrics
 - Change metrics

38

Scope of Software Metrics/9

Evaluation of methods and tools

- Efficiency of methods
- Efficiency and reliability of tools
- Certification test of acquired tools and components
- Benchmarking



49

39

Scope of Software Metrics/10

Capability maturity assessment

- US Software Engineering Institute (SEI) model (1989): CMM grading using five-level scale.
- **ISO 9001**: Quality systems: models for quality assurance in design/development, production, installation and servicing (1991)
- **ISO 9000-3**: Guidelines for application of ISO 9001 to the development, supply and maintenance of software (1991)



50

40

How to Implement?

- The eight steps required to implement a software measurement program are:
 - Document the software development process
 - State the goals
 - Define metrics required to reach goals
 - Identify data to collect
 - Define data collection procedures
 - Assemble a metrics toolset
 - Create a metrics database
 - Define the feedback mechanism

51

41

Who Benefits From Measurement

- **Managers**
 - What does each process cost?
 - How productive is the staff?
 - How good is the code being developed?
 - Will the user be satisfied with the product?
 - How can we improve?
- **Engineers**
 - Are the requirements testable?
 - Have we found all the failures?
 - Have we met our product or process goals?
 - What can we predict about our software product in the future?

52

42

Exercise

- Suppose that you are asked to study various software development tools and recommend the best three to your company. The following table shows a list of available development tools.
- *Work in Team 3 Students*

53

43

Exercise (cont'd)

Tool Name/Vendor	Languages Supported	Platforms	Features
Bean Machine IBM	Java	Windows, OS2, Unix	Best: Visual applet and JavaBean generation
CodeWarrior Pro Metrowerks	Java, C, C++, Pascal	Unix, Windows, Mac	Best: if you need to support Unix, Windows, and Mac platforms
Java Workshop Sun Microsystems	Java	Solaris, Windows	Better: Written 100% in Java; tools based on a web browser metaphor
JBuilder Imprise	Java	Windows, AS400	Better: database support
Visual Cafe for Java Symantec	Java	Windows	Good: multithreaded debugger
VisualAge IBM	Java	Unix, Windows	Good: includes incremental compiler and automatic version control
Visual J++ Microsoft	Java	Windows	Fair: All the bells and whistles for Windows

54

44

Exercise (cont'd)

- What are the *entities*, *attributes* and their *values* in your model?

<i>Entity</i>	<i>Attribute</i>	<i>Value</i>
Development Tool	Language supported	Java, C, C++, Pascal
	Platform	Win, Unix, Mac, OS2, AS400
	Feature	Fair, Good, Better, Best

55

45

Conclusion

- Without measurements there is no way to determine if the process/product are improving.
- Metrics allow the establishment of meaningful goals for improvement. **A baseline from which improvements can be measured can be established.**
- Metrics allow us to identify the causes of defects which have major effect on software development.
- When metrics are applied to a product they help identify:
 - which user requirements are likely to change
 - which modules are most error prone
 - how much testing should be planned for each module

56

46

Readings and Assignment 1

- You are required to watch 02 videos in the link below and Answer the following questions. The answers will be submitted on e-Learning system (Assignment 1) before 17:00, 03-03, 2025
- https://www.youtube.com/watch?v=bnydxXPN_rI
(overview SM)
- <https://www.youtube.com/watch?v=jiOaywcVssQ>
(Software Metrics)
- Questions
 - How important software measures are?
 - List at least 5 software metrics and short description for each.
 - What are the characteristics of software metrics?